

CRICKET CARD MARKETPLACE: TECHNICAL BRIEF

Users open packs to pull uniquely serialised cricket cards, then collect, list and trade them for a virtual currency called Runs. The MVP runs end to end on a local machine. This brief covers what exists, how the core mechanics are built, what was deliberately left out, and where we would like advice.

AUTHOR

Shreyansh Tiwari

DATE

26 September 2026

STATUS

Working MVP · local only · not deployed

CODEBASE

~3,500 lines (Python, SQL, TypeScript)

CONTENTS

01 Product

02 MVP status

03 Architecture

04 Data model

05 Core mechanics

06 Invariants

07 Economy

08 API

09 Frontend

10 Design decisions

11 Known gaps

12 Questions

13 Running it

01 PRODUCT

Crease is a digital trading-card game built around real cricket players. It is a personal project and a learning exercise in transactional backend design, so correctness under concurrency matters more here than feature count.

- **Card definitions** are templates: a player, a rarity (Common, Rare, Epic or Legendary) and a fixed print run (`max_supply`).

- **Card instances** are the owned objects. Each has a serial number that is unique within its definition, such as Rahul Dravid Legendary #003 of 5.
- **Packs** cost Runs and contain 3 randomly rolled cards. The purchase price is destroyed, not paid to anyone.
- **The market** uses fixed-price listings. A sale moves Runs from buyer to seller and burns a 5% fee.
- **Runs** are purely virtual. They cannot be bought with or cashed out for real money. New users receive 1,000 Runs.

A frozen long-term design exists, covering Postgres, MongoDB and Redis, a bid/ask order book and commit-reveal fair RNG. The build so far is a deliberately leaner Postgres-only version of it. Section 10 lists each divergence and the reason for it.

02 MVP STATUS

TABLES	ENDPOINTS	UI PAGES	AUTOMATED TESTS
8	21	4	0

ROADMAP PHASE	SCOPE	STATE
1 · Core data model	Players, card definitions, card instances, users	● DONE
2 · Marketplace	List, cancel and buy with an atomic transfer of Runs and ownership; currency ledger	● DONE
3 · Packs	Pack types, secure RNG, supply caps, atomic serial allocation	● DONE
4 · History	Append-only ownership event log, provenance and price history endpoints	● DONE
Frontend	React app covering packs, collection, market and card detail	● DONE
5 · Production maturity	Auth, tests, CI, pagination, indexes, deployment	● NOT STARTED

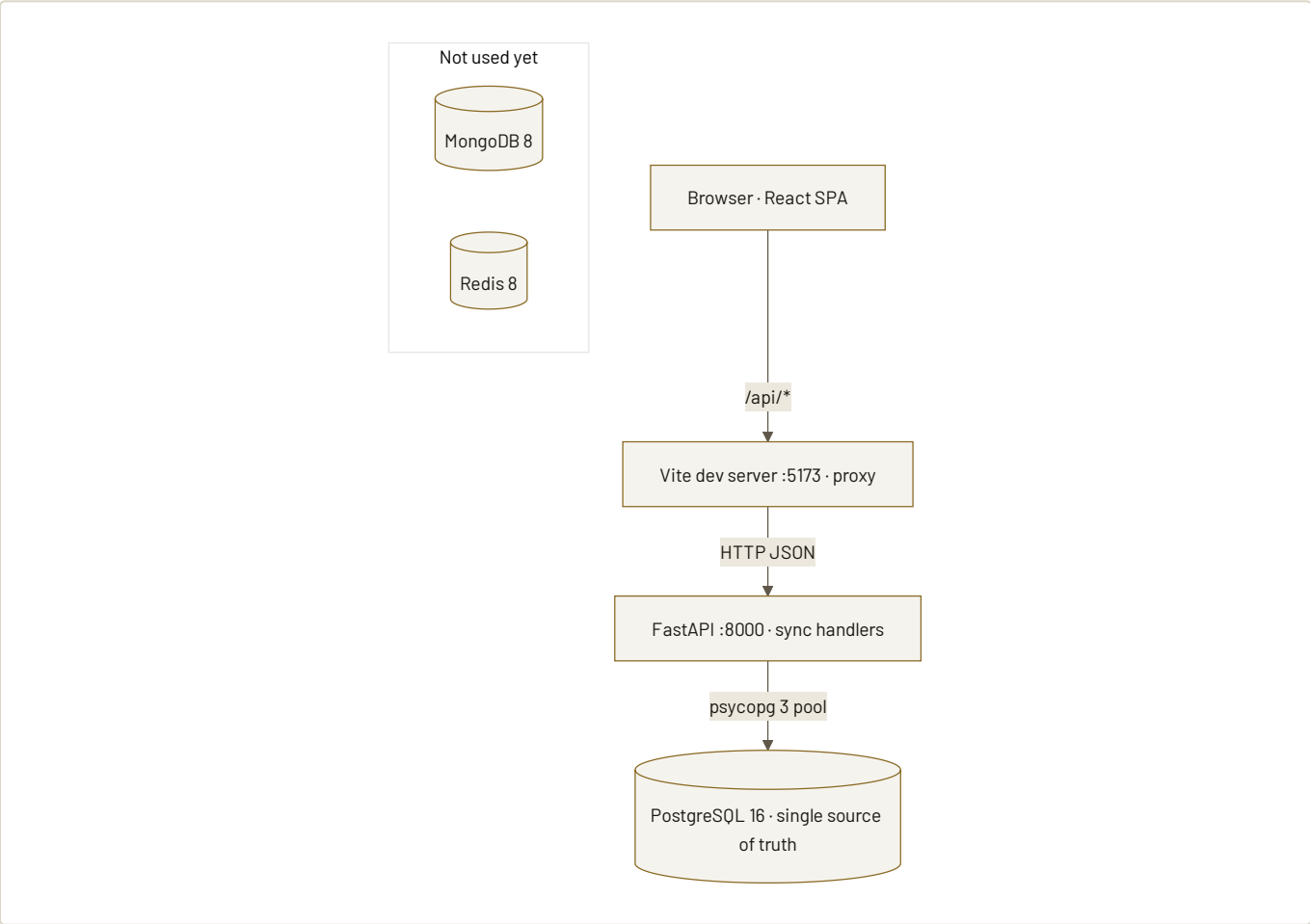
WHAT A USER CAN DO TODAY

- Open a *Club Pack* (100 Runs) or a *Test Pack* (300 Runs). The reveal is animated, and cards flip one at a time with the best card last.

- Browse their collection (the "Vault"), filtered by rarity and sorted by rarity or recency.
- List a card at a fixed price, with the fee and net proceeds shown before confirming, and cancel a listing.
- Browse the market with rarity filters and see the floor price, then buy with a confirm step.
- View any card's full provenance (pulled, listed, sold, delisted) and the sale history for its definition.
- Switch the user they are acting as from the header. This stands in for authentication, which does not exist yet.

03 ARCHITECTURE

The app is a modular monolith with a single FastAPI process and a single Postgres database. MongoDB and Redis containers are defined in `docker-compose.yml` but the application does not use them.



LAYER	CHOICE	NOTES
API	Python · FastAPI · Pydantic v2	Synchronous route handlers, which run in FastAPI's thread pool. Hand-written SQL, no ORM.
DB access	psycopg 3 + psycopg_pool	Rows come back as dicts. The pool's connection context manager commits on success and rolls back on exception, which is how every <code>HTTPException</code> raised mid-transaction undoes the partial work.

LAYER	CHOICE	NOTES
Database	PostgreSQL 16 (Docker)	Money is <code>NUMERIC(12,2)</code> . Ids are <code>BIGSERIAL</code> .
Schema	<code>schema.sql</code> + <code>migrations/*.sql</code>	Raw SQL applied by hand with <code>psql</code> . There is no migration tool or version tracking yet.
Frontend	React 19 · Vite 8 · TypeScript · Tailwind v4	TanStack Query for server state, React Router, and Motion for animation.
Fonts	Big Shoulders Display · Barlow · IBM Plex Mono	Self-hosted through <code>@fontsource</code> .

CODE LAYOUT

```
backend/  
  main.py          all routes + transactional helpers (~1,060 lines)  
  schemas.py       Pydantic request/response models  
  packs.py         pack config, odds and rarity rolling (pure, no DB)  
  database.py      connection pool  
  schema.sql       full schema for a fresh database  
  migrations/  
    001_packs.sql   mint counter, pack_openings, constraints  
    002_ownership_events.sql event log + backfill from existing data  
frontend/src/  
  api.ts session.tsx toast.tsx format.ts index.css  
  components/ TradingCard CardBack Pack Layout  
  pages/      PacksPage CollectionPage MarketPage CardPage
```

04 DATA MODEL

There are two denormalised "current state" columns: `users.balance` and `card_instances.owner_id`. Each is backed by an append-only log that records how it got its value. Both columns are always written in the same transaction as their log row.

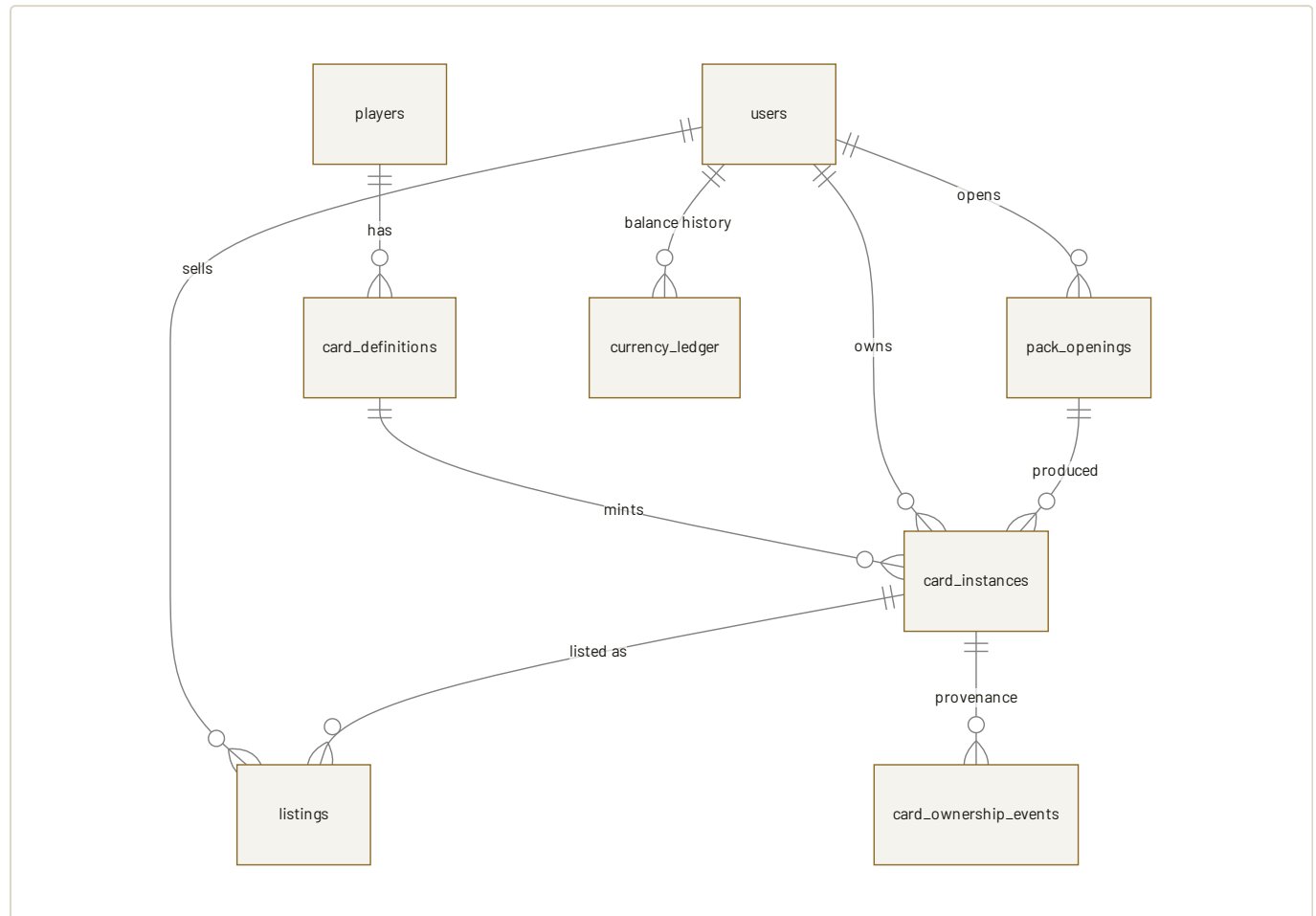


TABLE	KEY COLUMNS	CONSTRAINTS THAT CARRY WEIGHT
<code>users</code>	username, email, <code>balance</code>	<code>CHECK (balance >= 0)</code> , the last line of defence against overdraft
<code>players</code>	name, country, role	none
<code>card_definitions</code>	player_id, rarity, <code>max_supply</code> , <code>minted_count</code>	<code>minted_count <= max_supply</code> , <code>max_supply > 0</code> , rarity limited to the four values
<code>card_instances</code>	definition, <code>serial_number</code> , <code>owner_id</code> , <code>pack_opening_id</code>	<code>UNIQUE (card_definition_id, serial_number)</code>
<code>listings</code>	card, seller, price, status, <code>resolved_at</code>	Status is ACTIVE, SOLD or CANCELLED. A partial unique index on <code>card_instance_id</code> WHERE <code>status = 'ACTIVE'</code> allows at most one active listing per card.
<code>pack_openings</code>	user, pack_type, price	<code>price > 0</code>

TABLE	KEY COLUMNS	CONSTRAINTS THAT CARRY WEIGHT
<code>currency_ledger</code>	user, <code>delta</code> , reason, related listing or pack	<code>delta <> 0</code> ; reason limited to an enumerated set (section 5.4)
<code>card_ownership_events</code>	card, event_type, from and to users, price, related ids	Event type is MINTED, PULLED, LISTED, DELISTED or SOLD. Indexed on (<code>card_instance_id</code> , <code>created_at</code>).

05 CORE MECHANICS

Each mutating operation is a single database transaction. Correctness comes from Postgres row locks plus guarded `UPDATE ... WHERE` statements whose affected row count is checked. There are no application-level or distributed locks.

5.1 BUYING A LISTING

- 01 `SELECT ... FROM listings WHERE id = ? FOR UPDATE`, then reject unless the status is ACTIVE. A concurrent buyer blocks here and then sees SOLD.
- 02 Lock both user rows **in ascending id order**, so two transactions touching the same pair can never deadlock. Check that the buyer exists and has enough balance.
- 03 Run a guarded transfer: `UPDATE card_instances SET owner_id = buyer WHERE id = ? AND owner_id = seller`. If zero rows change, the listing is stale and the transaction aborts before any money moves.
- 04 Append a SOLD event to the ownership log.
- 05 Record the money movement: `TRANSFER_PURCHASE_DEBIT` (-price, buyer), `TRANSFER_SALE_CREDIT` (+price, seller), then `BURN_MARKET_FEE` (-5%, seller).
- 06 `UPDATE listings SET status = 'SOLD' WHERE id = ? AND status = 'ACTIVE'`, then commit.

Cancelling a listing locks the listing row the same way and guards on `status = 'ACTIVE'`. Before that guard existed, a cancel racing a buy could overwrite SOLD with CANCELLED. Creating a listing locks the card row while it checks ownership, which closes a race where a seller could list a card they had just sold.

5.2 OPENING A PACK

- 01 Lock the user row and check their balance.
- 02 Take an **unlocked snapshot** of every definition with supply remaining, then roll all 3 slots in memory (`packs.py`). Each slot rolls a rarity with `secrets.SystemRandom`, then picks a definition uniformly within that rarity. If a rarity is sold out, the slot falls back to the nearest rarity with supply, preferring the lower one. The snapshot is decremented locally so a single pack cannot over-pick a card.
- 03 Insert the `pack_openings` row and burn the price (`BURN_PACK_PURCHASE`).

- 04 Mint each picked card in **ascending definition-id order**, which is the deadlock-avoidance rule for concurrent packs.
- 05 If any mint finds its definition sold out, because another pack took the last copy after the snapshot, the endpoint returns 409 and the whole transaction rolls back, charge included.

5.3 SERIAL NUMBERS AND SUPPLY CAPS

Every card, whether pulled from a pack or minted through the admin endpoint, goes through one helper that runs:

```
UPDATE card_definitions
SET minted_count = minted_count + 1
WHERE id = %s AND minted_count < max_supply
RETURNING minted_count; -- this value becomes the serial number
```

This one statement row-locks the definition so concurrent mints queue up, hands out the next serial, and enforces the print run. There is no read-max-plus-one in application code. The `UNIQUE (definition, serial)` constraint backs it up.

5.4 CURRENCY LEDGER

All balance changes go through `apply_balance_change()`, which updates `users.balance` and inserts the ledger row together. Nothing else in the codebase writes to `balance`. The reason prefix encodes the kind of movement:

CATEGORY	REASONS	RULE
MINT_*	SIGNUP, DEV_GRANT	Currency created from nothing; no offsetting row. The only way Runs enter the system.
BURN_*	PACK_PURCHASE, MARKET_FEE	Currency destroyed; no offsetting credit anywhere. A burn that credited someone would be a duplication bug.
TRANSFER_*	PURCHASE_DEBIT, SALE_CREDIT	Always an equal-and-opposite pair with the same listing id, written in the same transaction.

5.5 OWNERSHIP EVENT LOG

`record_card_event()` is called from every path that creates a card, moves it, or changes its listing state, inside the same transaction. Migration 002 rebuilt history for cards that existed before the log. `listings` never stored a buyer, but the buyer can be derived: cards only change hands through sales, so the buyer is the seller of the card's next listing, or its current owner if it hasn't been listed since.

5.6 LOCK ORDERING

OPERATION	ROWS LOCKED, IN ORDER
Buy	listing → users (ascending id) → card_instance
Cancel listing	listing
Create listing	card_instance
Open pack	user → card_definitions (ascending id)
Dev grant	user

We believe no cycle exists across these paths. A second opinion is welcome (section 12).

06 INVARIANTS AND VERIFICATION

These invariants were checked by hand with parallel `curl` requests plus SQL audit queries against the live dev database. None of this is automated yet.

INVARIANT	HOW IT WAS CHECKED	RESULT
Exactly one buy succeeds per listing	Two simultaneous buy requests on the same listing	● 1 × 200, 1 × 409
Serials are unique and gap-free within each definition	20 packs opened concurrently by 2 users; per definition, <code>count = max(serial) = distinct serials = minted_count</code>	● ALL DEFINITIONS PASS
No deadlocks under concurrent pack opening	Same 20-pack run	● 20 / 20 SUCCEEDED
Every pack contains exactly 3 cards	Group cards by pack	● 21 PACKS, 0 BAD
<code>SUM(ledger.delta) = balance</code> for every user	Audit query after every test run	● 0 MISMATCHES
Owner equals the <code>to_user</code> of the latest MINTED, PULLED or SOLD event	Lateral-join audit across all cards	● 68 / 68 MATCH
Supply cap is enforced	Mint against a definition with <code>max_supply = 1</code> that is already full	● 409 RETURNED

07 ECONOMY PARAMETERS

All of these are constants in code. None are stored or versioned in the database.

PACK	PRICE	CARDS	COMMON	RARE	EPIC	LEGENDARY
Club Pack(<code>standard</code>)	100	3	70%	22%	7%	1%
Test Pack(<code>premium</code>)	300	3	40%	35%	18%	7%

- **Sources:** a 1,000-Run signup grant and a dev-only faucet endpoint. The daily login grant from the design is not built.
- **Sinks:** the entire pack price, plus a 5% market fee rounded half-up to 0.01 and burned from the seller's proceeds.
- **Seed print runs:** Common 500, Rare 100, Epic 25, Legendary 5 to 10, across 6 Indian players.
- The odds apply to each card independently; packs have no guaranteed-rarity slots.

08 API SURFACE

There is no authentication. The acting user is passed explicitly as `buyer_id` , `seller_id` or `user_id` . Errors use FastAPI's `{"detail": ...}` shape: 404 for missing entities, 409 for state conflicts, 403 for ownership violations, and 400 or 422 for validation.

	PATH	PURPOSE
<code>GET</code>	<code>/health</code>	Liveness
<code>GET</code>	<code>/users</code>	List users with balances
<code>POST</code>	<code>/users</code>	Create a user (+1,000 Runs, <code>MINT_SIGNUP</code>)
<code>POST</code>	<code>/dev/users/{id}/grant</code>	Dev-only faucet (<code>MINT_DEV_GRANT</code>)
<code>GET</code>	<code>/players</code> , <code>/players/{id}</code>	Player catalogue
<code>POST</code>	<code>/players</code>	Create a player
<code>GET</code>	<code>/card-definitions</code>	Catalogue with <code>minted_count</code>
<code>POST</code>	<code>/card-definitions</code>	Create a definition (rarity and supply are validated)
<code>GET</code>	<code>/card-definitions/{id}/price-history</code>	Recent SOLD events across all serials
<code>POST</code>	<code>/card-instances</code>	Admin mint through the shared counter
<code>GET</code>	<code>/card-instances</code> , <code>/card-instances/{id}</code>	Card detail joined with player and owner
<code>GET</code>	<code>/card-instances/{id}/history</code>	Provenance timeline
<code>GET</code>	<code>/users/{id}/cards</code>	Collection, including each card's active listing

	PATH	PURPOSE
GET	/marketplace	Active listings sorted by price; filters for definition, player and rarity
POST	/listings	Create a listing
POST	/listings/{id}/cancel?seller_id=	Cancel your own listing
POST	/listings/{id}/buy?buyer_id=	Atomic purchase (5.1)
GET	/packs	Pack types, prices and odds
POST	/packs/open	Atomic pack opening (5.2)

09 FRONTEND

- **Visual direction:** a premium dark look in the style of NBA Top Shot or FIFA Ultimate Team, with a palette drawn from cricket: night-match black, trophy brass, cricket-whites cream and pitch green. Rarity colours are chalk, ball-leather red, emerald and gold, avoiding the generic purple "epic".
- **Card art is typographic and line-art** (stumps and creases), not player photos. This avoids image-rights questions and keeps cards consistent.
- **Card effects:** 3D tilt, a glare that follows the pointer, and a foil layer on Epic and Legendary cards. Pointer position is written straight to CSS custom properties, not React state, so tracking never triggers a re-render. All inner sizes use container-query units (`cqw`), so one component renders correctly at thumbnail and hero size.
- **Pack reveal:** a shake held for at least 1.1 s while the request resolves, then a tear animation and a face-down deal sorted so the best card comes last. Rare-or-better cards glow before they are flipped, and Epic and Legendary reveals trigger a particle burst.
- **Server state:** TanStack Query, with the affected queries invalidated after every mutation.
- **Session:** a "Playing as" user picker persisted in `localStorage`, standing in for authentication.
- **Dev proxy:** the SPA calls `/api/*` and Vite forwards it to FastAPI, so the browser sees a single origin and CORS is never involved in development.

10 DESIGN DECISIONS AND DIVERGENCES

Each row lists what the original design called for, what was built instead, and why. We would welcome views on any of them.

AREA	ORIGINAL DESIGN	BUILT	REASON
Card content store	MongoDB for rolled stats, traits, flavour text	Postgres only; cards have no rolled stats yet	No flexible-schema need yet. Mongo is deferred until per-instance stats arrive.
Cache and pub/sub	Redis for order-book cache, rate limits, live events	Not used	No read pressure and no WebSockets yet
Serial numbers	Uncapped at first	Capped per definition from day one	The schema already had <code>max_supply</code> , and the guarded counter enforces it at no extra cost
Money type	BIGINT integer Runs	<code>NUMERIC(12,2)</code>	Matched the existing column. Integer Runs would be simpler, since fractional Runs only come from the 5% fee.
Ids	UUID	BIGSERIAL	Simpler; there are no multi-writer or merge needs
Ledger system side	Nullable <code>user_id</code> for the system side	<code>user_id</code> NOT NULL; mint and burn rows are single-sided	Every row so far touches a real user
Trading	Bid/ask order book, matching engine	Fixed-price listings only	Phase 3 of the original plan; intentionally deferred
Fair RNG	Commit-reveal with server and client seeds	Server-side CSPRNG only	Planned for Phase 4
Deployment	Split into services later	Monolith	Matches the design's own guidance for the MVP

11 KNOWN GAPS AND RISKS

GAP	IMPACT	SEVERITY
No authentication. The acting user id is a request parameter.	Anyone can act as anyone. This blocks any public deployment.	● HIGH
The dev faucet endpoint is open	Unlimited currency minting	● HIGH
No automated tests or CI	The concurrency guarantees in section 6 are verified by hand only and could regress silently	● HIGH
Postgres data lives in an anonymous Docker volume	<code>docker compose down</code> followed by <code>up</code> would lose all data	● HIGH
Nothing since the initial commit is committed to git	All of this work exists only in the working tree	● HIGH
Database credentials hard-coded in <code>database.py</code> ; no config or env layer	Fine locally; not deployable as is	● MEDIUM
Migrations are raw SQL with no version tracking	Easy to apply twice or skip one	● MEDIUM
No pagination on list endpoints; few indexes beyond primary keys	Collection, market and history scans grow linearly	● MEDIUM
No rate limiting on opening packs or buying	Easy to script or spam	● MEDIUM
Pack odds are not versioned or recorded per opening	Cannot prove which odds applied to a past pull	● MEDIUM
<code>main.py</code> is one ~1,060-line module	Harder to navigate; no routers or service layer	● LOW
The buy and cancel endpoints take the acting user as a query parameter	Inconsistent with the JSON bodies used elsewhere	● LOW
Uncommitted, unused <code>pymongo</code> import in <code>database.py</code>	Opens a Mongo client at import time for no reason	● LOW

12 QUESTIONS FOR REVIEWERS

What should come next?

The candidates are auth; automated concurrency tests with CI; per-card rolled stats (the trigger for adding MongoDB); a bid/ask order book; or deployment. What order would you choose, and why?

Is the locking model sound?

Are the lock orders in 5.6 deadlock-free across all paths? Is Read Committed plus row locks and guarded updates enough, or should any path run at Serializable?

Pack opening under contention.

Picks come from an unlocked snapshot, and the pack fails with 409 if a card sells out mid-transaction. Is fail-and-retry the right behaviour, or should the server re-roll the affected slot?

Currency representation.

Should we switch to integer Runs (`BIGINT`) now, while the data is disposable?

Tooling.

Should we adopt Alembic or another migration tool? Should we keep hand-written SQL or move to SQLAlchemy Core, async psycpg, or an ORM?

Is MongoDB still justified?

The original design admits JSONB would work for card content. Would you keep the Postgres + Mongo split, or put rolled stats in JSONB?

Economy tuning.

Do the prices, odds, 5% fee and print runs look like they would produce a healthy market? What should we instrument before real users arrive?

Auth approach.

For a hobby-scale app, which would you recommend: sessions, JWT, or a hosted provider?

Deployment.

What is the smallest sensible production setup: one VM with Docker Compose, or managed Postgres plus a PaaS?

13 RUNNING IT LOCALLY

```
# 1. Databases (only Postgres is used)
docker start cricket-postgres          # or: docker compose up -d

# 2. Schema (fresh database only; existing ones apply migrations/ in order)
docker exec -i cricket-postgres psql -U cricket -d cricket_marketplace < backend/schema.sql

# 3. API (from the repo root)
.venv/bin/uvicorn backend.main:app --port 8000 --reload --reload-dir backend

# 4. Frontend
cd frontend && npm install && npm run dev    # http://localhost:5173

# API docs (FastAPI Swagger): http://localhost:8000/docs
```

The dev database currently holds 4 users, 6 players, 13 card definitions, 77 card instances, 25 pack openings, 11 listings, 42 ledger rows and 94 ownership events, all generated during manual testing.